



---

# Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

|                    |                              |
|--------------------|------------------------------|
| NIM                | 71251221                     |
| Nama Lengkap       | RIZKY FEBRIANTO KRISTIANSYAH |
| Minggu ke / Materi | 15 / Fungsi Rekursif         |

PROGRAM STUDI INFORMATIKA  
FAKULTAS TEKNOLOGI INFORMASI  
UNIVERSITAS KRISTEN DUTA WACANA  
YOGYAKARTA  
2026

## **BAGIAN 1: Materi**

### **Pengertian Rekursif**

Fungsi rekursif adalah fungsi yang berisi atau mendefinisikan dirinya sendiri, sehingga sering disebut sebagai fungsi yang memanggil dirinya sendiri. Fungsi rekursif merupakan fungsi matematis yang berulang dan memiliki pola yang terstruktur. Namun demikian, fungsi ini perlu diperhatikan agar dapat berhenti dengan benar dan tidak menghabiskan memori sistem. Fungsi rekursif merupakan fungsi yang harus digunakan secara hati-hati karena dapat bersifat unlimited loop sehingga menyebabkan program hang up.

Fungsi rekursif akan terus berjalan sampai kondisi berhenti terpenuhi. Oleh karena itu, dalam sebuah fungsi rekursif perlu terdapat dua blok penting, yaitu blok yang menjadi titik berhenti dari proses rekursif dan blok yang memanggil dirinya sendiri. Di dalam rekursif terdapat dua bagian utama. Pertama adalah Base Case, yaitu bagian yang menentukan kapan fungsi rekursif berhenti berjalan. Kedua adalah Rekursif Case, yaitu bagian yang berisi statement yang akan terus diulang-ulang hingga mencapai Base Case. Tanpa adanya Base Case yang tepat, fungsi rekursif akan berjalan tanpa henti dan menyebabkan error berupa stack overflow atau program hang.

### **Kelebihan dan Kekurangan**

Fungsi rekursif memiliki sejumlah keunggulan dibandingkan pendekatan iteratif. Pertama, kode program yang dihasilkan lebih singkat dan elegan karena logika yang kompleks dapat diungkapkan dalam beberapa baris saja. Kedua, masalah yang kompleks dapat dipecah menjadi sub-masalah yang lebih kecil di dalam rekursif sehingga lebih mudah dipahami secara konseptual. Pendekatan divide and conquer yang menjadi dasar rekursif sangat cocok digunakan untuk permasalahan yang memiliki struktur berulang seperti faktorial, fibonacci, dan konversi basis bilangan.

Di sisi lain, fungsi rekursif juga memiliki beberapa kelemahan yang perlu dipertimbangkan. Pertama, fungsi rekursif memakan memori yang lebih besar karena setiap kali fungsi memanggil dirinya sendiri, dibutuhkan sejumlah ruang memori tambahan pada call stack untuk menyimpan state dari pemanggilan sebelumnya. Kedua, fungsi rekursif mengorbankan efisiensi dan kecepatan eksekusi dibandingkan dengan pendekatan iteratif untuk kasus yang sama. Ketiga, fungsi rekursif lebih sulit dilakukan proses debugging dan kadang sulit dimengerti alirannya, terutama ketika terdapat banyak level pemanggilan yang terjadi secara bersamaan.

## Bentuk Umum dan Studi Kasus

Bentuk umum fungsi rekursif pada Python terdiri dari definisi fungsi yang di dalamnya terdapat pemanggilan kembali terhadap fungsi itu sendiri. Setiap fungsi rekursif pasti memiliki solusi iteratifnya, namun pendekatan rekursif seringkali lebih ringkas dan lebih dekat dengan definisi matematisnya. Sebagai studi kasus pertama adalah penghitungan faktorial, yaitu perkalian deret angka dari 1 hingga  $n$ . Secara matematis, faktorial dapat didefinisikan sebagai:  $\text{fact}(n) = 1$  jika  $n = 0$ , dan  $\text{fact}(n) = n * \text{fact}(n-1)$  jika  $n > 0$ . Definisi matematis inilah yang secara langsung diterjemahkan menjadi fungsi rekursif dalam Python.

**Pseudocode (recursive):**

```
function factorial is:  
input: integer  $n$  such that  $n \geq 0$   
output:  $[n \times (n-1) \times (n-2) \times \dots \times 1]$   
  
1. if  $n$  is 0, return 1  
2. otherwise, return  $[n \times \text{factorial}(n-1)]$   
  
end factorial
```

Gambar 1.1: Pseudocode faktorial

```
1 def faktorial(n):  
2     if n==0 or n==1:  
3         return 1  
4     else:  
5         return faktorial(n-1) * n  
6  
7 print(faktorial(4))
```

Gambar 1.2: Sourcecode faktorial menggunakan fungsi rekursif

Proses perhitungan yang terjadi pada  $\text{faktorial}(4)$  adalah sebagai berikut: fungsi pertama kali dipanggil dengan  $n=4$ , karena 4 bukan 0 atau 1 maka fungsi memanggil  $\text{faktorial}(3)$  dan seterusnya hingga mencapai Base Case yaitu  $n=1$  yang mengembalikan nilai 1. Setelah Base Case tercapai, nilai dikembalikan ke atas secara bertahap: 1 dikembalikan, lalu dikalikan 2 menjadi 2, dikalikan 3 menjadi 6, dan terakhir dikalikan 4 menghasilkan 24. Pola pemanggilan ini menggambarkan bagaimana Rekursif Case terus memanggil dirinya sendiri dengan parameter yang semakin mendekati Base Case.

Studi kasus kedua adalah perkalian dua bilangan menggunakan metode penjumlahan berulang secara rekursif. Misalnya  $2 * 4 = 2 + 2 + 2 + 2 = 8$ . Logika rekursifnya adalah:

perkalian(bil1, bil2) = bil1 jika bil2 == 1, dan perkalian(bil1, bil2) = bil1 + perkalian(bil1, bil2-1) jika bil2 > 1. Base Case terjadi ketika bil2 bernilai 1, di mana hasilnya langsung adalah bil1 itu sendiri. Studi kasus ketiga adalah deret Fibonacci, yaitu deret bilangan di mana setiap bilangan merupakan penjumlahan dua bilangan sebelumnya: 1, 1, 2, 3, 5, 8, 13, 21, 34, ... Rumus rekursifnya adalah  $F(n) = 1$  jika  $n=1$  atau  $n=2$ , dan  $F(n) = F(n-1) + F(n-2)$  jika  $n > 2$ .

## BAGIAN 2: LATIHAN MANDIRI

Link Github repository: [https://github.com/ifgodwillsit/71251221\\_febrina](https://github.com/ifgodwillsit/71251221_febrina)

### Latihan 15.1

```
def is_prima(n, i = 2):  
    if n < 2:  
        return False  
    if i * i > n:  
        return True  
    if n % i == 0:  
        return False  
    return is_prima(n, i + 1)  
  
n = int(input("Masukkan bilangan: "))  
if is_prima(n):  
    print(n, "adalah bilangan prima")  
else:  
    print(n, "bukan bilangan prima")
```

Gambar 2.1: Source code Latihan 15.1

Konsep latihan ini adalah mengecek apakah suatu bilangan termasuk bilangan prima menggunakan fungsi rekursif. Program meminta user input integer. Base case pertama adalah jika  $n < 2$  maka bukan prima, base case kedua adalah jika  $i * i > n$  maka semua pembagi sudah dicek dan bilangan tersebut prima. Recursive case berjalan dengan mengecek apakah  $n$  habis dibagi  $i$ , jika ya berarti bukan prima, jika tidak maka fungsi memanggil dirinya sendiri dengan  $i + 1$ .

```
Masukkan bilangan: 17  
17 adalah bilangan prima
```

Gambar 2.2: Output Latihan 15.1

## Latihan 15.2

```
def is_palindrom(kalimat):  
    kalimat = kalimat.replace(' ', '').lower()  
    if len(kalimat) <= 1:  
        return True  
    if kalimat[0] != kalimat[-1]:  
        return False  
    return is_palindrom(kalimat[1:-1])  
  
kalimat = input('Masukkan kalimat: ')  
if is_palindrom(kalimat):  
    print('"' + kalimat + '" adalah palindrom')  
else:  
    print('"' + kalimat + '" bukan palindrom')
```

Gambar 2.3: Sourcecode Latihan 15.2

Konsep latihan ini adalah mengecek apakah suatu kalimat merupakan palindrom (terbaca sama dari depan atau belakang) menggunakan fungsi rekursif. Sebelum pengecekan, spasi dihapus dan semua huruf diubah ke lowercase agar perbandingan tidak case sensitive. Base casenya jika panjang string  $\leq 1$  maka sudah pasti palindrom. Rekursif Case mengecek apakah karakter pertama dan terakhir sama, jika tidak sama maka bukan palindrom, jika sama maka fungsi memanggil dirinya sendiri dengan string yang sudah dipotong karakter pertama dan terakhirnya menggunakan slicing `[1:-1]`.

```
Masukkan kalimat: abang naba  
"abang naba" adalah palindrom
```

Gambar 2.4: Output Latihan 15.2

## Latihan 15.3

```
def deret_ganjil(n):  
    if n == 1:  
        return 1  
    else:  
        return (2 * n - 1) + deret_ganjil(n - 1)  
  
n = int(input('Masukkan jumlah suku (n): '))  
print('Jumlah deret ganjil:', deret_ganjil(n))
```

Gambar 2.5: Sourcecode Latihan 15.3

Konsep latihan ini adalah menghitung jumlah deret bilangan ganjil  $1 + 3 + 5 + 7 + \dots$  hingga suku ke- $n$  menggunakan fungsi rekursif. Bilangan ganjil ke- $n$  dapat dinyatakan dengan rumus  $(2*n - 1)$ , sehingga suku ke-1 adalah 1, suku ke-2 adalah 3, suku ke-3 adalah 5, dan seterusnya sampai  $n$ . Base casenya ketika  $n = 1$  yang mengembalikan nilai 1 (bilangan ganjil pertama). Recursive case menghitung bilangan ganjil ke- $n$  menggunakan rumus  $(2*n - 1)$  dan menjumlahkannya dengan hasil pemanggilan rekursif untuk  $n-1$ , terus berulang hingga mencapai base case.

```
Masukkan jumlah suku (n): 3
Jumlah deret ganjil: 9
```

Gambar 2.6: Output Latihan 15.3

#### Latihan 15.4

```
def jumlah_digit(n):
    n = abs(n)
    if n < 10:
        return n
    else:
        return (n % 10) + jumlah_digit(n // 10)

n = int(input('Masukkan bilangan: '))
print('Jumlah digit:', jumlah_digit(n))
```

Gambar 2.7: Sourcecode Latihan 15.4

Konsep latihan ini adalah menghitung jumlah dari seluruh digit dari suatu bilangan menggunakan fungsi rekursif. Misalnya bilangan 234 maka jumlah digitnya adalah  $2 + 3 + 4 = 9$ . Fungsi `abs()` digunakan di awal untuk menangani bilangan negatif. Base case adalah ketika  $n < 10$ , artinya bilangan sudah satu digit dan langsung dikembalikan nilainya. Recursive case mengambil digit terakhir menggunakan operasi modulo ( $n \% 10$ ), kemudian menjumlahkannya dengan hasil pemanggilan rekursif untuk sisa bilangan yang sudah dibuang digit terakhirnya menggunakan pembagian bulat ( $n // 10$ ). Proses ini berulang hingga bilangan tersisa hanya satu digit.

```
Masukkan bilangan: 234
Jumlah digit: 9
```

Gambar 2.8: Output Latihan 15.4

#### Latihan 15.5

```
def faktorial(n):
    if n == 0 or n == 1:
        return 1
    return faktorial(n - 1) * n

def kombinasi(n, r):
    if r == 0 or r == n:
        return 1
    return kombinasi(n - 1, r - 1) + kombinasi(n - 1, r)

n = int(input('Masukkan nilai n: '))
r = int(input('Masukkan nilai r: '))
print('C(' + str(n) + ', ' + str(r) + ') =', kombinasi(n, r))
```

Gambar 2.9: Sourcecode Latihan 15.5

Konsep latihan ini adalah menghitung kombinasi  $C(n, r)$  menggunakan fungsi rekursif. Rumus kombinasi adalah  $C(n, r) = n! / (r! * (n-r)!)$ . Pendekatan rekursif yang digunakan

memanfaatkan identitas Pascal yaitu  $C(n, r) = C(n-1, r-1) + C(n-1, r)$ . Base case terjadi ketika  $r = 0$  atau  $r = n$ , di mana keduanya selalu menghasilkan nilai 1 karena hanya ada satu cara memilih 0 elemen atau memilih semua elemen. Recursive case memecah permasalahan  $C(n, r)$  menjadi dua sub-permasalahan yang lebih kecil dan menjumlahkan hasilnya, terus berulang hingga mencapai salah satu base case.

```
Masukkan nilai n: 5  
Masukkan nilai r: 2  
C(5,2) = 10
```

Gambar 2.10: Output Latihan 15.5